

Mandatory: This document includes dedicated Benefits sections (§2, §11, §12) for strategic, operational, technical, business, stakeholder, and phased rollout value.

Industry Prime (FMS)

Agentic AI & RAG Architecture Guide

Document version: 1.0

Date: May 2026

Platform: Industry Prime — React + FastAPI + Supabase

Purpose: How to add Agentic AI with a RAG pipeline, develop AI separately, and sync Support plus other data into an AI knowledge database.

Table of Contents

1. [Executive Summary](#)
 2. [Benefits \(Mandatory Overview\)](#)
 3. [What Is Agentic AI vs RAG](#)
 4. [Where AI Helps in Industry Prime](#)
 5. [How Agentic AI Works \(Architecture\)](#)
 6. [RAG Pipeline — Step by Step](#)
 7. [Develop AI Individually + Multi-Database Sync](#)
 8. [AI Database Design](#)
 9. [Security & Compliance](#)
 10. [Phased Rollout](#)
 11. [Benefits by Stakeholder](#)
 12. [Benefits by Phase](#)
 13. [Costs, Risks & Mitigations](#)
 14. [Next Steps & Decisions](#)
-

1. Executive Summary

Industry Prime today runs on **React (Vite)**, **FastAPI**, and **Supabase PostgreSQL** for support tickets, KPI dashboards, client payment, training, and checklists. There is **no built-in Agentic AI or RAG** yet.

This guide describes how to:

- Add **Agentic AI** (planning + tool use on your existing APIs).
- Build a **RAG pipeline** (retrieve answers from your own tickets, docs, and rules).
- **Develop the AI layer separately**, then **pull data** from the Support database and other module tables into a dedicated **AI database** (vector index + text chunks).

Operational data remains the source of truth; the AI database is a search-optimized copy for assistants and agents.

2. Benefits (Mandatory Overview)

2.1 Strategic benefits

Benefit	Description
Faster support resolution	Staff find similar resolved chores/bugs and past solutions in seconds instead of manual Register search.
Consistent answers	RAG grounds responses in <code>quality_solution</code> and internal docs, reducing inconsistent replies across operators.
Lower training burden	New users ask natural-language questions (“How do I filter Payment Ageing?”) instead of memorizing every screen.
Scalable knowledge	Every resolved ticket and updated doc automatically enriches the knowledge base via sync jobs.
Safe evolution	AI is built beside the main app; production tickets and payments are unchanged until you enable write tools with approval.

2.2 Operational benefits

Benefit	Description
Less repetitive work	Agents draft summaries, similar-ticket lists, and stage suggestions; humans approve.
Better use of existing data	Support, payment ageing, KPI, and training data become searchable without new manual wikis.

Cross-module insights	One assistant can combine ticket context with payment/KPI tools (with role permissions).
Incremental delivery	Ship Support-only RAG first, then payment/KPI sync—each phase delivers value.

2.3 Technical benefits

Benefit	Description
Reuse current APIs	Agents call existing GET /tickets, payment-ageing, KPI routes—no duplicate business logic.
Clear separation	AI service + ingest jobs + AI tables are isolated from core CRUD flows.
Incremental sync	ai_sync_state tracks last pull; only changed rows are re-embedded.
Auditability	Chunks store source_system, source_id, and metadata for citations in the UI.

2.4 Business benefits

Benefit	Description
Higher customer satisfaction	Faster, accurate responses on chores and bugs.
Reduced SLA breaches	Quicker identification of similar fixes and bottlenecks.
Better management visibility	Natural-language queries over KPI and payment patterns (admin roles).
Future-ready platform	Foundation for approval assistants, auto-tagging, and smart dashboards.

3. What Is Agentic AI vs RAG

Type	What it does	Best for
Simple AI chat	One question → one answer (may be generic).	Generic help only.

RAG (Retrieval-Augmented Generation)	Embeds your data; answers use retrieved chunks (tickets, docs).	"How was GRN issue fixed before?"
Agentic AI	LLM plans steps, calls tools (your APIs), checks results, retries.	"List overdue chores for Company X and draft a summary."

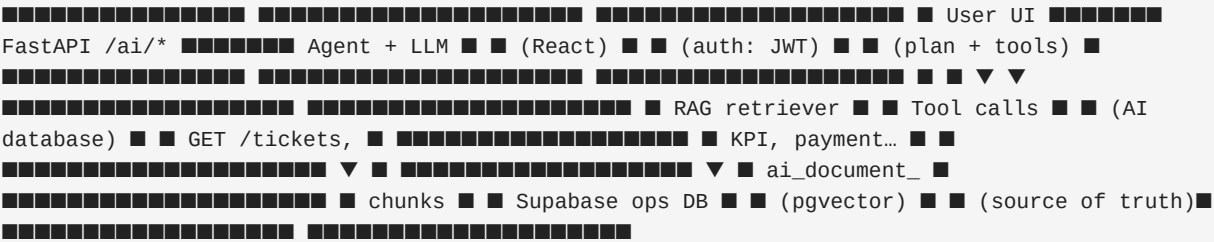
Recommended for Industry Prime: RAG for knowledge + Agentic layer for read-only (then approved) actions on existing APIs.

4. Where AI Helps in Industry Prime

Module	Agentic + RAG use
Support tickets (Chores, Bugs, Features)	Similar resolved issues; why a ticket is stuck; draft replies from past <code>quality_solution</code> .
Register of Tickets / Solutions	Semantic search over solutions, not only exact text match.
Performance / Success KPI	Explain NA status, pillar numbers (with links to dashboard).
Client Payment / Payment Ageing	"Which clients exceed X days in Q1?" via tools + payment SOP docs in RAG.
Onboarding / Training / Checklist	"What is pending for Company X?" from training/checklist summaries.
Dashboard / Support dashboard	Natural language: "Pending bugs this week for [company]."
Internal ops	Copilot over docs/ runbooks and SLA rules.

Use with caution (human approval): payment writes, role changes, ticket status changes.

5. How Agentic AI Works (Architecture)



Principles

1. Same **JWT / roles** as today; AI never bypasses section permissions.
2. **Tools** wrap existing FastAPI endpoints—not raw SQL from the LLM.
3. **Human in the loop** for any write (draft → user confirms).
4. **Never index** secrets (.env, service keys, full PII tables).

6. RAG Pipeline — Step by Step

Step 1 — Choose sources to index

Source	Content to chunk	Update trigger
Resolved tickets	description, quality_solution, company, page	On complete / nightly
Open tickets (optional)	description, status fields	Hourly
docs/*.md	Runbooks, payment/KPI notes	On deploy / manual
Payment ageing (summary)	Aggregates + labels, not raw bank data	Nightly
KPI / performance	Labels, NA reasons (filtered)	Nightly

Step 2 — Ingestion (offline)

```
Documents → split (500-800 tokens) → embed → upsert ai_document_chunks
```

- **Embeddings:** e.g. OpenAI `text-embedding-3-small` or open-source model on Render.
- **Store:** Supabase **pgvector** table.

Step 3 — Retrieval

1. Embed user question.
2. Vector similarity search (`ORDER BY embedding <=> query_embedding LIMIT k`).
3. Filter by user's allowed companies / roles.

Step 4 — Generation

- Build prompt: system rules + retrieved chunks + user question.

- LLM answers with **citations** (reference_no, doc path).

Step 5 — Agentic tools (optional)

- Tools: search_similar_tickets, get_ticket, list_open_chores, get_payment_ageing_report (role-gated).
- Loop: LLM → tool call → result → LLM until final answer (max steps capped).

Step 6 — Frontend

- Route e.g. “Ask IP Assistant” in header or Support.
- Show sources under each answer.

7. Develop AI Individually + Multi-Database Sync

7.1 Develop separately

Component	Location	Role
AI API	backend/app/ai/ or ip-ai-service	Chat, RAG, agent
Ingestion	backend/app/ai/sync/	ETL from Supabase → AI DB
AI DB	Supabase (pgvector)	Chunks + embeddings only
UI	/ai-assistant (optional)	Chat drawer

Main app screens stay unchanged until you integrate the assistant.

7.2 Pull from Support and other data

All modules today share **one Supabase Postgres**. “Other database” = **other tables**, synced with different source_system values.

source_system	Source tables / data	Indexed content
support	tickets, remarks	Problems, solutions, refs
payment	invoices, ageing API	Summaries, quarter labels
kpi	performance_monitoring	Pillar labels, NA text

training	training / checklist	Task summaries
docs	repo markdown	How-to and SOPs

7.3 Sync pattern (incremental)

```
1. Read ai_sync_state.last_synced_at for source_system
2. SELECT rows WHERE updated_at > last_synced_at
3. Build text → chunk → embed → upsert chunks (by source_id)
4. Update ai_sync_state
5. On delete/clear: remove chunks for that source_id
```

7.4 Two modes at query time

Mode	Use
Pre-indexed RAG	History, docs, past solutions
Live agent tools	“Right now” open queue, today’s KPI

Best practice: Combine both.

8. AI Database Design

```
-- Searchable knowledge
CREATE TABLE ai_document_chunks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_system TEXT NOT NULL, -- support | payment | kpi | training | docs
  source_table TEXT,
  source_id TEXT,
  chunk_index INT DEFAULT 0,
  content TEXT NOT NULL,
  embedding VECTOR(1536),
  metadata JSONB,
  updated_at TIMESTAMPTZ DEFAULT NOW() );
-- Incremental sync cursor
CREATE TABLE ai_sync_state (
  source_system TEXT PRIMARY KEY,
  last_synced_at TIMESTAMPTZ,
  last_cursor TEXT );
```

Enable **pgvector** in Supabase; add indexes for `source_system` and vector search.

9. Security & Compliance

- Sync jobs run **server-side only** (service role, not browser).
- **RLS** on `ai_document_chunks` aligned with ticket/company access.
- Exclude confidential companies if required.
- Log prompts/responses; optional disable per role.
- Do not index passwords, tokens, or full payment PII.

10. Phased Rollout

Phase	Deliverable	Benefits delivered
1	AI DB + Support sync + RAG Q&A	Faster similar-ticket lookup; grounded answers
2	Docs sync + UI assistant	Self-serve how-to; less training time
3	Payment/KPI sync	Cross-module questions for admins
4	Agent read-only tools	Live queue + historical context in one chat
5	Draft writes with approval	Draft replies without auto-changing tickets

11. Benefits by Stakeholder

Stakeholder	Key benefits
Support operators	Find past solutions fast; draft responses; fewer escalations
Approvers / admins	Summaries across tickets; policy-aligned answers from docs
Payment / accounts team	Quick ageing questions with citations to report logic
KPI / Success team	Explain metrics and NA rules in plain language
Management	Visibility via natural language; no new BI tool for simple questions
IT / developers	Modular AI service; no monolith rewrite; reuse existing APIs

12. Benefits by Phase

Phase	Benefits
Phase 1 (Support RAG)	Immediate ROI on chore/bug resolution; uses data you already have

Phase 2 (Docs + UI)	Lower onboarding time for new staff
Phase 3 (Payment/KPI)	Single assistant across modules (permission-aware)
Phase 4 (Agent tools)	Real-time accuracy plus historical depth
Phase 5 (Approved writes)	Time saved on data entry without losing control

13. Costs, Risks & Mitigations

Item	Notes
Cost	Embedding + LLM per query; cache frequent questions
Risk: wrong answer	Require citations; “verify in dashboard” for KPI/payment
Risk: data leak	RLS + field allowlists on sync
Risk: scope creep	Start Support-only; add sources one by one

14. Next Steps & Decisions

Before implementation, confirm:

1. **AI DB location:** Same Supabase project vs separate project?
2. **First source:** Support resolved tickets only?
3. **Sync schedule:** Hourly vs nightly?
4. **LLM provider:** OpenAI / Azure OpenAI / Gemini?
5. **Read-only vs approved writes** for v1?

Document control

Field	Value
Title	Agentic AI & RAG Architecture — Industry Prime

Benefits section	§2, §11, §12 (mandatory)
Related systems	Support, Client Payment, KPI, Training, Supabase

End of document